

```

1 # Import important libraries
2 import pandas as pd
3 import numpy as np
4
5 import matplotlib.pyplot as plt
6 import seaborn as sns
7
8 from sklearn.preprocessing import StandardScaler
9 from sklearn.cluster import KMeans
10 from sklearn.ensemble import IsolationForest
11
12 from sklearn.metrics import confusion_matrix
13 from sklearn.metrics import classification_report

```

```

1 df = pd.read_csv('creditcard.csv')
2 df

```

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255412
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387071
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817724
...	...	...	...	...	...	...	...	...	...	...
284802	172786.0	-11.881118	10.071785	-9.834783	-2.066656	-5.364473	-2.606837	-4.918215	7.305334	1.914421
284803	172787.0	-0.732789	-0.055080	2.035030	-0.738589	0.868229	1.058415	0.024330	0.294869	0.584806
284804	172788.0	1.919565	-0.301254	-3.249640	-0.557828	2.630515	3.031260	-0.296827	0.708417	0.432451
284805	172788.0	-0.240440	0.530483	0.702510	0.689799	-0.377961	0.623708	-0.686180	0.679145	0.392081
284806	172792.0	-0.533413	-0.189733	0.703337	-0.506271	-0.012546	-0.649617	1.577006	-0.414650	0.486181

284807 rows x 31 columns

Q1 What is fraud detection?

Answer:

Fraud detection is the process of detecting abnormal and suspicious transactions that may have been unauthorized. Financial institutions use these processes to protect customers from fraud and reduce financial loss.

Q2 Why is it difficult?

Answer:

Fraud detection is challenging because fraudulent transactions are very rare compared to legitimate ones. Fraudsters also constantly change their methods, making fraud patterns difficult to identify.

Q3 Why use unsupervised learning?

Answer:

Unsupervised learning is useful when transaction labels (fraud or non-fraud) are unavailable or limited. Instead of learning from labeled examples, the algorithm identifies unusual patterns and anomalies in the data. This makes it effective for detecting new or previously unseen types of fraud.

```

1 # Display first 5 rows
2 df.head()

```

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...

5 rows × 31 columns

```
1 # Check number of rows and columns
2 df.shape
```

```
(284807, 31)
```

Observation

The dataset contains 284,807 transactions and 31 columns.

```
1 # Display dataset information
2 df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
#   Column      Non-Null Count  Dtype
---  -
0    Time        284807 non-null  float64
1    V1           284807 non-null  float64
2    V2           284807 non-null  float64
3    V3           284807 non-null  float64
4    V4           284807 non-null  float64
5    V5           284807 non-null  float64
6    V6           284807 non-null  float64
7    V7           284807 non-null  float64
8    V8           284807 non-null  float64
9    V9           284807 non-null  float64
10   V10          284807 non-null  float64
11   V11          284807 non-null  float64
12   V12          284807 non-null  float64
13   V13          284807 non-null  float64
14   V14          284807 non-null  float64
15   V15          284807 non-null  float64
16   V16          284807 non-null  float64
17   V17          284807 non-null  float64
18   V18          284807 non-null  float64
19   V19          284807 non-null  float64
20   V20          284807 non-null  float64
21   V21          284807 non-null  float64
22   V22          284807 non-null  float64
23   V23          284807 non-null  float64
24   V24          284807 non-null  float64
25   V25          284807 non-null  float64
26   V26          284807 non-null  float64
27   V27          284807 non-null  float64
28   V28          284807 non-null  float64
29   Amount       284807 non-null  float64
30   Class        284807 non-null  int64
dtypes: float64(30), int64(1)
memory usage: 67.4 MB
```

```
1 # Check data types
2 print(df.dtypes)
```

```
Time        float64
V1          float64
V2          float64
V3          float64
V4          float64
V5          float64
```

```
V6      float64
V7      float64
V8      float64
V9      float64
V10     float64
V11     float64
V12     float64
V13     float64
V14     float64
V15     float64
V16     float64
V17     float64
V18     float64
V19     float64
V20     float64
V21     float64
V22     float64
V23     float64
V24     float64
V25     float64
V26     float64
V27     float64
V28     float64
Amount  float64
Class   int64
dtype: object
```

Observation:

All features are numerical. Therefore, no categorical encoding is required before applying machine learning algorithms.

```
1 # Check missing values
2 print(df.isnull().sum())
```

```
Time      0
V1        0
V2        0
V3        0
V4        0
V5        0
V6        0
V7        0
V8        0
V9        0
V10       0
V11       0
V12       0
V13       0
V14       0
V15       0
V16       0
V17       0
V18       0
V19       0
V20       0
V21       0
V22       0
V23       0
V24       0
V25       0
V26       0
V27       0
V28       0
Amount    0
Class     0
dtype: int64
```

```
1 # Total missing values
2 print("Total Missing Values:", df.isnull().sum().sum())
```

```
Total Missing Values: 0
```

Observation:

The dataset contains zero missing values, so no data imputation is required.

```
1 # Generate statistical summary
2 df.describe()
```

	Time	V1	V2	V3	V4	V5	V6
count	284807.000000	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05
mean	94813.859575	1.168375e-15	3.416908e-16	-1.379537e-15	2.074095e-15	9.604066e-16	1.487313e-15
std	47488.145955	1.958696e+00	1.651309e+00	1.516255e+00	1.415869e+00	1.380247e+00	1.332271e+00
min	0.000000	-5.640751e+01	-7.271573e+01	-4.832559e+01	-5.683171e+00	-1.137433e+02	-2.616051e+01
25%	54201.500000	-9.203734e-01	-5.985499e-01	-8.903648e-01	-8.486401e-01	-6.915971e-01	-7.682956e-01
50%	84692.000000	1.810880e-02	6.548556e-02	1.798463e-01	-1.984653e-02	-5.433583e-02	-2.741871e-01
75%	139320.500000	1.315642e+00	8.037239e-01	1.027196e+00	7.433413e-01	6.119264e-01	3.985649e-01
max	172792.000000	2.454930e+00	2.205773e+01	9.382558e+00	1.687534e+01	3.480167e+01	7.330163e+01

8 rows x 31 columns

Observation

The summary provides information about mean, standard deviation, minimum and maximum values of all features.

```
1 # Count fraud and non-fraud transactions
2 df['Class'].value_counts()
```

	count
Class	
0	284315
1	492

dtype: int64

Observation

Class 0 represents normal transactions while Class 1 represents fraudulent transactions. The dataset is highly imbalanced.

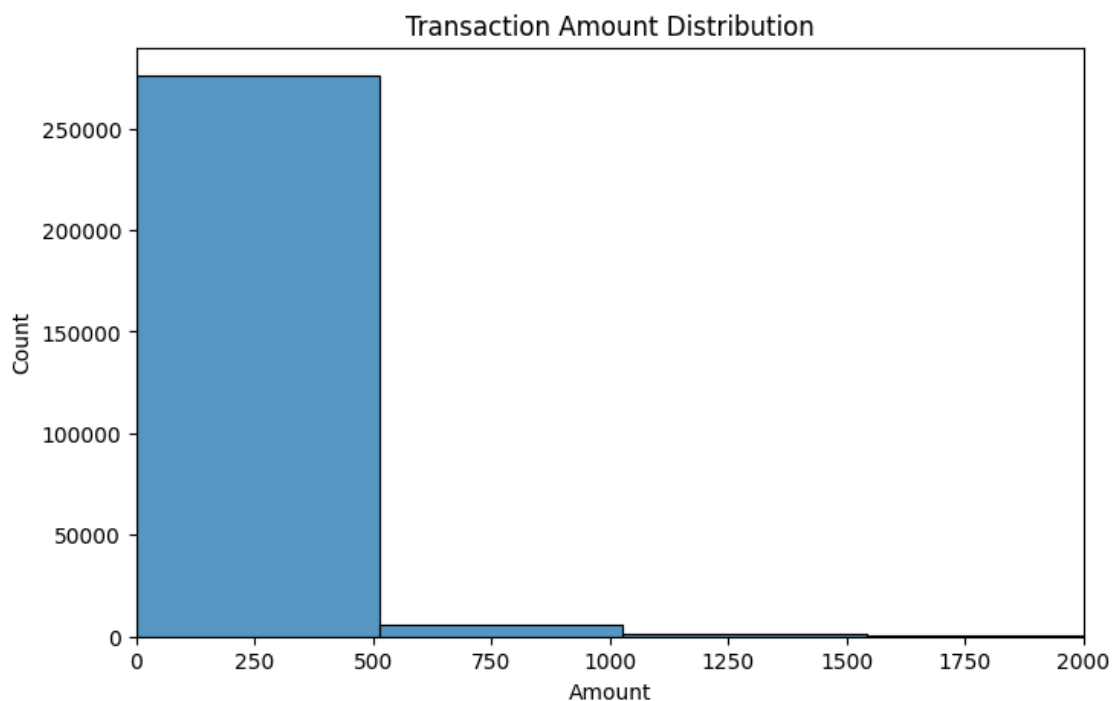
```
1 # Plot fraud vs non-fraud transactions
2 sns.countplot(x='Class', data=df)
3 plt.title("Fraud vs Non-Fraud Transactions")
4 plt.show()
```



Observation

Normal transactions are significantly higher than fraudulent transactions.

```
1 # Distribution of transaction amounts
2 plt.figure(figsize=(8,5))
3 sns.histplot(df['Amount'], bins=50)
4 plt.title("Transaction Amount Distribution")
5 plt.xlim(0,2000)
6 plt.show()
```

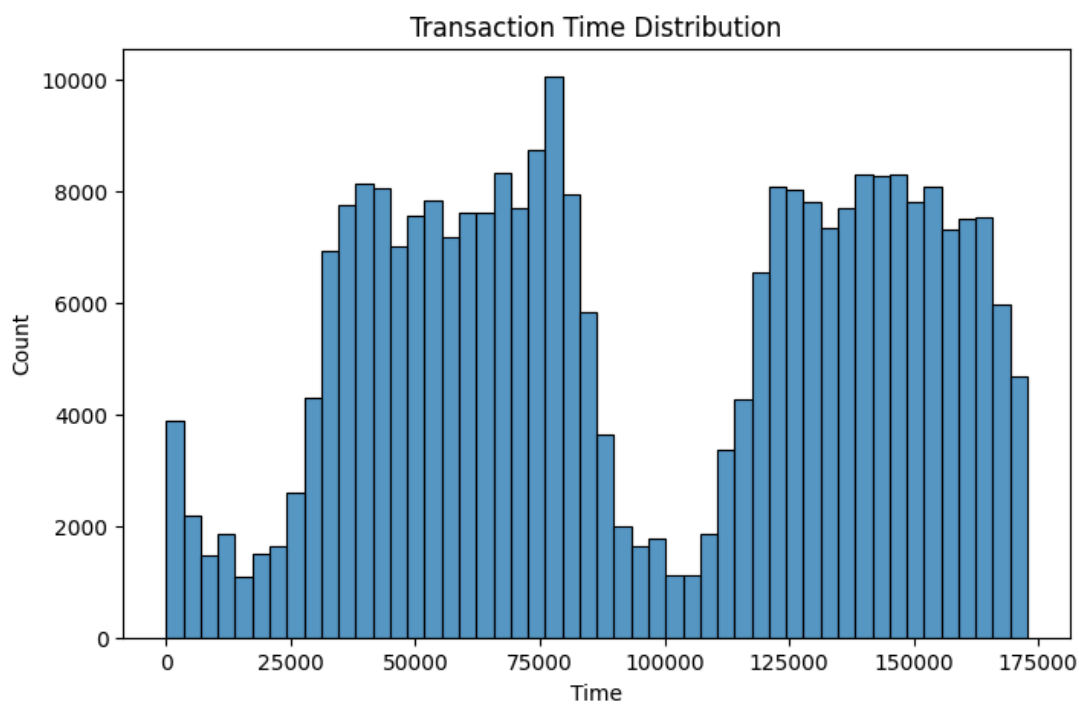


Observation

Most transactions have lower transaction amounts while high-value transactions are less frequent.

```
1 # Distribution of transaction time
2 plt.figure(figsize=(8,5))
3 sns.histplot(df['Time'], bins=50)
```

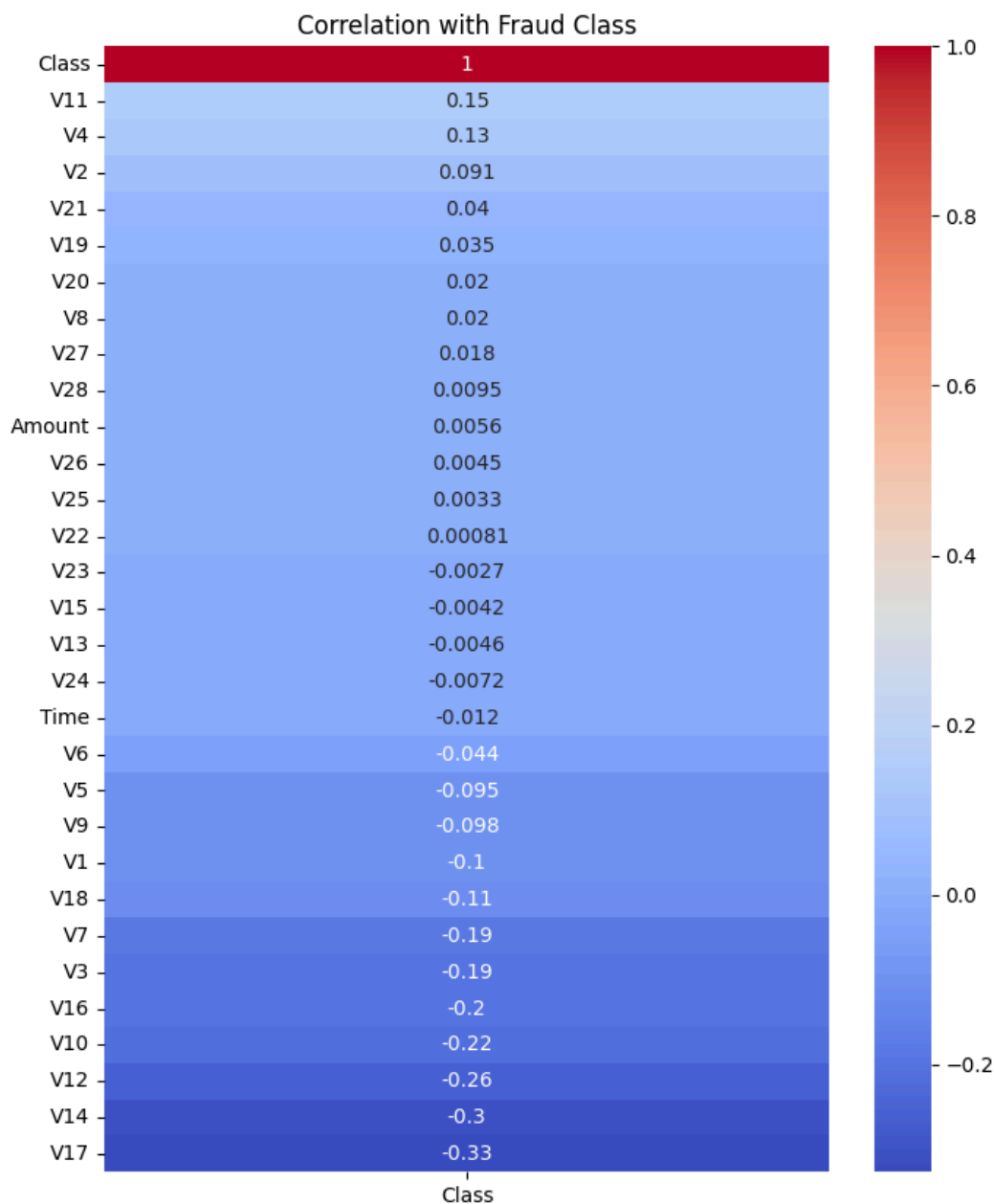
```
4 plt.title("Transaction Time Distribution")
5 plt.show()
```



Observation

Transactions occur throughout the entire recorded period with varying frequencies.

```
1 # Correlation with target variable
2 corr = df.corr()['Class'].sort_values(ascending=False)
3 plt.figure(figsize=(8,10))
4 sns.heatmap(
5     corr.to_frame(),
6     annot=True,
7     cmap='coolwarm'
8 )
9 plt.title("Correlation with Fraud Class")
10 plt.show()
```



Observation

Some variables show stronger positive or negative relationships with fraud transactions.

```
1 # Separate features and target
2 X = df.drop('Class', axis=1)
3 y = df['Class']
```

```
1 # Standardize features
2 scaler = StandardScaler()
3 X_scaled = scaler.fit_transform(X)
4 print("Scaling Completed")
```

Scaling Completed

```
1 # Create K-Means model
2 kmeans = KMeans(
3     n_clusters=2,
4     random_state=42
5 )
6 # Train model and predict clusters
7 clusters = kmeans.fit_predict(X_scaled)
```

```
1 # Store cluster labels
2 df['Cluster'] = clusters
3 df[['Cluster']].head()
```

	Cluster
0	0
1	0
2	0
3	0
4	0

```
1 # Number of transactions in each cluster
2 df['Cluster'].value_counts()
```

Cluster	count
0	155337
1	129470

dtype: int64

Observation

Transactions are divided into two clusters by the K-Means algorithm.

```
1 # Create cross-tabulation
2 cluster_table = pd.crosstab(
3     df['Cluster'],
4     df['Class']
5 )
6 cluster_table
```

Class	0	1
0	155038	299
1	129277	193

Next steps: [Generate code with cluster_table](#) [New interactive sheet](#)

Observation

This helps identify which cluster contains more fraud cases.

```
1 # Find cluster with maximum fraud cases
2 fraud_cluster = cluster_table[1].idxmax()
3 print("Fraud Cluster:", fraud_cluster)
```

Fraud Cluster: 0

```
1 # Mark fraud cluster as fraud
2 pred_kmeans = (
3     clusters == fraud_cluster
4 ).astype(int)
```

```
1 # Evaluate K-Means
2 print(confusion_matrix(y, pred_kmeans))
```



```
[[129277 155038]
 [   193   299]]
```

Observation

The confusion matrix shows the number of correctly and incorrectly classified transactions.

```
1 # Detailed performance report
2 print(classification_report(
3     y,
4     pred_kmeans
5 ))
```

	precision	recall	f1-score	support
0	1.00	0.45	0.62	284315
1	0.00	0.61	0.00	492
accuracy			0.45	284807
macro avg	0.50	0.53	0.31	284807
weighted avg	1.00	0.45	0.62	284807

Observation

K-Means generates a large number of false positives due to class imbalance.

```
1 # Create Isolation Forest model
2 iso = IsolationForest(
3     contamination=0.002,
4     random_state=42
5 )
6 # Train model
7 pred_iso = iso.fit_predict(X_scaled)
```

```
1 # Convert output:
2 # -1 = anomaly (fraud)
3 # 1 = normal
4 pred_iso = np.where(
5     pred_iso == -1,
6     1,
7     0
8 )
```

```
1 # Evaluate Isolation Forest
2 print(confusion_matrix(
3     y,
4     pred_iso
5 ))
```

```
[[283883  432]
 [   354  138]]
```

Observation

Isolation Forest identifies anomalies with fewer false positives than K-Means.

```
1 # Detailed performance report
2 print(classification_report(
3     y,
4     pred_iso
5 ))
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	284315
1	0.24	0.28	0.26	492

accuracy			1.00	284807
macro avg	0.62	0.64	0.63	284807
weighted avg	1.00	1.00	1.00	284807

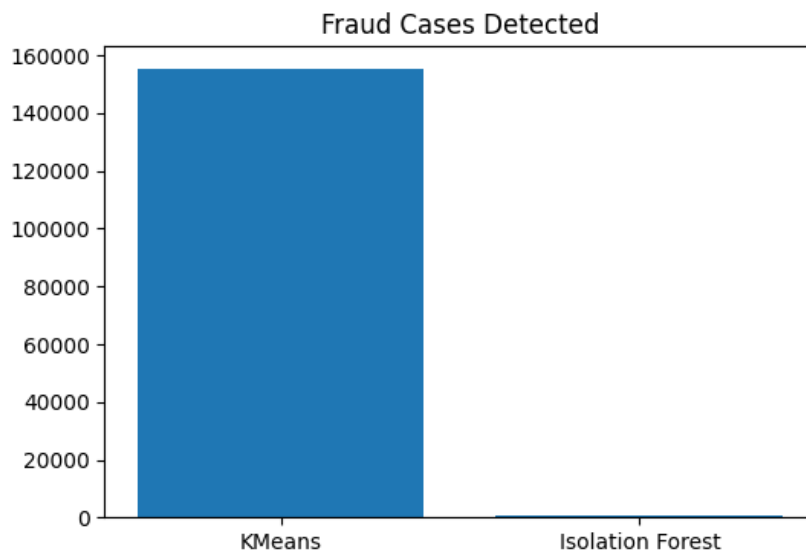
Observation

Isolation Forest performs better for anomaly detection because it isolates unusual observations.

```
1 # Count predicted frauds
2 print("Actual Frauds:", y.sum())
3 print("KMeans Detected:", pred_kmeans.sum())
4 print("Isolation Forest Detected:", pred_iso.sum())
```

```
Actual Frauds: 492
KMeans Detected: 155337
Isolation Forest Detected: 570
```

```
1 # Compare detected fraud cases
2 models = [
3     "KMeans",
4     "Isolation Forest"
5 ]
6 frauds_detected = [
7     pred_kmeans.sum(),
8     pred_iso.sum()
9 ]
10 plt.figure(figsize=(6,4))
11
12 plt.bar(
13     models,
14     frauds_detected
15 )
16 plt.title("Fraud Cases Detected")
17 plt.show()
```



```
1 # Comparison between KMeans algorithm and Isolation Forest
2 comparison = pd.DataFrame({
3     'Model': ['KMeans', 'Isolation Forest'],
4     'Detected Frauds': [pred_kmeans.sum(), pred_iso.sum()]
5 })
6 comparison
```

	Model	Detected Frauds	
0	KMeans	155337	
1	Isolation Forest	570	